

Project Report

Group Name: Aura of Turku

Group Description: 919

Group Members: Sabuj Bhowmick, Shaktiman Choudhury

To make the step counter function, we based the application on the ble_app_uart example from the nrf_sdk template projects. Similarly to Exercise 5, we flashed the S132 SoftDevice hex file onto the board before running any code from SEGGER Embedded Studio. Then, in SEGGER, we wrote the code for lsm6ds0 setup, twi_init, and added the section for step count in the setup. We fixed the main function to call the necessary functions. We also wanted to debug where we were facing problems, so we added debug codes for each step to see in the nrf connect app logs where the device or code was getting stuck.

Our first major challenge was the "nrf_ble_gq.h: No such file or directory" issue. We fixed the issue by specifying the nrf_ble_gq inclusion path in the Preprocessor settings and manually adding the nrf_ble_gq.c source file to the project tree.

Then we got the "NRFX_TWI0_INST_IDX undeclared" problem. This occurred because the TWI (I2C) driver, which allows the nRF52 to communicate with the LSM6DSO, was deactivated by default. We fixed this by looking through sdk_config.h and enabling NRFX_TWI_ENABLED, NRFX_TWI0_ENABLED, and their legacy TWI equivalents. This enabled the compiler to finally "see" the hardware instance required by our twi_init() function.

The integration of the LSM6DSO driver was the most complicated element. We encountered many incompatible-pointer-type problems as a result of our platform_write function. After removing the write function and inserting the lsm6ds0 setup from the exercises, as well as the lsm6ds0_reg.c, we continued to receive "implicit declaration" issues for pedometer functions. We realized that different driver versions had different naming standards. We modified the code, referencing lsm6ds0_reg.h, to use lsm6ds0_embedded_sens_set to activate the pedometer and lsm6ds0_number_of_steps_get to get the data.

To complete the connection, we updated the main loop to read the step register every 1000 milliseconds. We used sprintf to transform the uint16_t step count into a formatted string, which we then transmitted using the ble_nus_data_send function. We added a status check with the m_nus_ready flag and the m_conn_handle to verify that the program only sends data once per second when a phone is connected, and the service is ready.

The Android-based nRF Connect app was used for verification. We connected to the device "PHW_StepCounter", found the Nordic UART Service (NUS), and activated notifications on the TX Characteristic. As the board moved, the "Steps: X" strings updated in real time when we pressed the downwards button inside the service, confirming that the sensor logic and BLE protocol had been successfully merged.